Heart Disease Risk Prediction

End-to-End MLOps Pipeline

Machine Learning Operations (AIMLCZG523)
Assignment 01 — 50 Marks

Submitted by

Pentela Veeranjineyulu

BITS Pilani — Work Integrated Learning Programme
M.Tech in Artificial Intelligence & Machine Learning

October 2026

Repository: https://github.com/veerusgithub/heart-mlops

Table of Contents

1. Introduction & Problem Statement

2. Data Acquisition & Exploratory Data Analysis [5 marks]

3. Feature Engineering & Model Development [8 marks]

4. Experiment Tracking with MLflow [5 marks]

5. Model Packaging & Reproducibility [7 marks]

6. CI/CD Pipeline & Automated Testing [8 marks]

7. Model Containerization with Docker [5 marks]

8. Production Deployment on Kubernetes [7 marks]

9. Monitoring & Logging [3 marks]

10. Architecture Overview

11. Setup & Installation Instructions

12. Conclusion

1. Introduction & Problem Statement

Cardiovascular diseases (CVDs) are the leading cause of death globally, accounting for an estimated 17.9 million lives each year according to the World Health Organization. Early detection and accurate risk assessment can significantly improve patient outcomes by enabling timely interventions. This project addresses this critical healthcare challenge by building a machine learning classifier that predicts heart disease risk from routine clinical measurements, and deploying it as a production-ready, monitored REST API.

The assignment implements a complete MLOps lifecycle — from raw data acquisition through exploratory analysis, leak-free preprocessing, experiment tracking, automated testing, CI/CD, Docker containerization, Kubernetes orchestration, and Prometheus + Grafana monitoring. Every component is wired together so the entire pipeline runs reproducibly from a clean checkout using a single requirements.txt file.

Problem Statement: Given 13 clinical features from the UCI Heart Disease dataset (Cleveland Clinic, 303 patients), build a binary classifier to predict the presence or absence of heart disease, package it as a FastAPI microservice, and deploy it on Kubernetes with full observability.

Key Design Principles:

(a) Reproducibility — fixed random seeds (42), pinned dependency ranges, and a data download script that reads a bundled copy of the original UCI file.

(b) Train/Serve Parity — preprocessing and the model live in a single scikit-learn Pipeline object, guaranteeing that an API request is transformed exactly like a training row, eliminating skew.

(c) Production-Readiness — non-root Docker container, liveness/readiness probes, input validation with Pydantic Literal types enforcing UCI codes, and a CI pipeline that fails on any code or test error.

2. Data Acquisition & Exploratory Data Analysis [5 marks]

2.1 Data Acquisition

The dataset is the UCI Heart Disease dataset, Cleveland Clinic subset (processed.cleveland.data), obtained from the UCI Machine Learning Repository. A reproducible download script (src/download_data.py) fetches the file, assigns proper column headers, and saves it as data/heart_raw.csv. The bundled copy of the original UCI file (heart+disease/processed.cleveland.data) ensures offline reproducibility.

Dataset Characteristics:

- Rows: 303 patients (Cleveland Clinic cohort)
- Features: 13 clinical attributes (age, sex, chest pain type, blood pressure, cholesterol, blood sugar, ECG, heart rate, exercise angina, ST depression, ST slope, fluoroscopy vessels, thalassemia)
- Target: Binary — num > 0 encoded as target = 1 (disease present). Original num ranges 0–4 representing different diagnoses.
- Missing values: 6 total (ca: 4 rows, thal: 2 rows) — marked as '?' in the raw file, handled by median/mode imputation inside the pipeline.
- Raw UCI encodings preserved: cp 1–4 (not 0–3), slope 1–3, thal 3/6/7 (normal, fixed defect, reversible defect), ca 0–3, restecg 0–2.

2.2 Data Cleaning

Cleaning is intentionally minimal and deterministic — operations that are safe to do before the train/test split (no information leakage). The basic_clean() function in src/preprocessing.py: (1) drops exact duplicate rows, (2) coerces all '?' entries to NaN using pd.to_numeric(errors='coerce'), and (3) casts the target to integer. All value-based imputation (median for numeric, most-frequent for categorical) is deferred to the sklearn ColumnTransformer so it is fit only on the training fold during cross-validation.

2.3 Exploratory Data Analysis

The EDA script (src/eda.py) generates four professional visualizations from the cleaned dataset. The class distribution shows 164 healthy vs 139 disease patients (45.9% positive rate), indicating a reasonably balanced dataset that does not require aggressive oversampling. Class weights are set to 'balanced' in both classifiers as a safeguard.

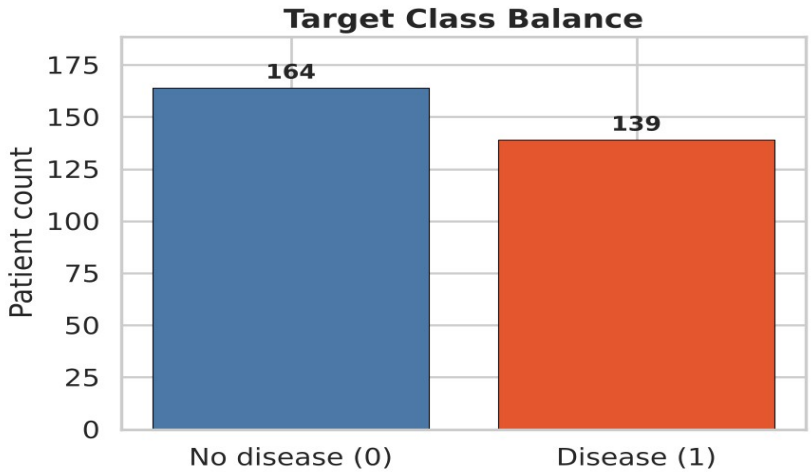


Figure 1: Class Balance — 164 healthy (54.1%) vs 139 disease (45.9%)

Feature distributions reveal clinically meaningful patterns. Age ranges from 29 to 77 with a concentration around 55–60, matching the known epidemiology of heart disease. Resting blood pressure (trestbps) clusters around 120–140 mm Hg with a right tail. Serum cholesterol (chol) centers near 250 mg/dl with notable outliers above 400. Maximum heart rate (thalach) shows a bimodal distribution that correlates inversely with disease presence.

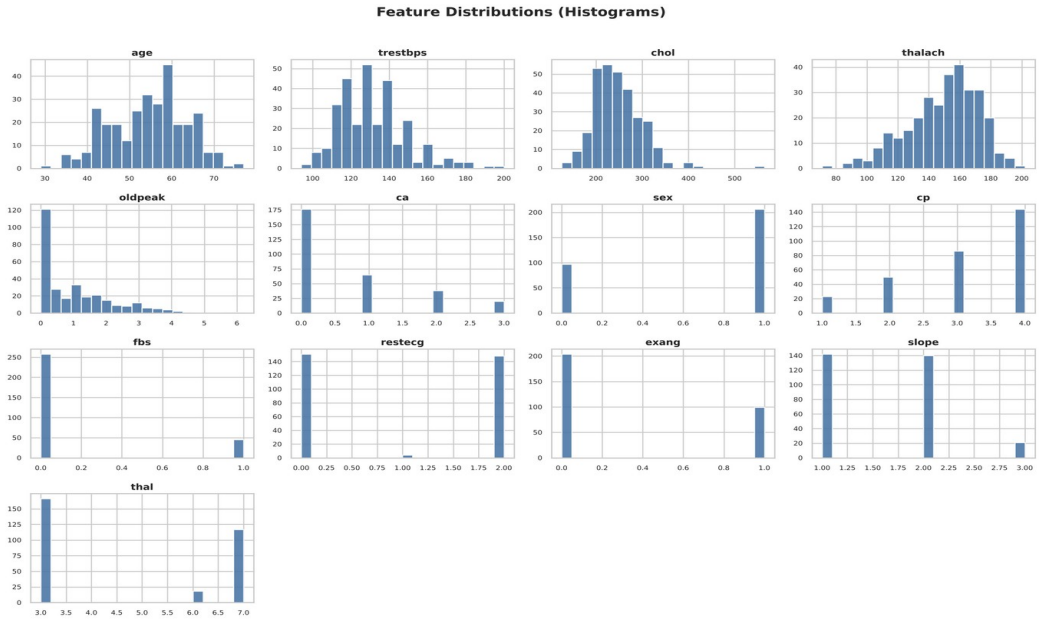


Figure 2: Feature Distributions — Histograms of all 13 clinical features

The correlation heatmap identifies the features most strongly associated with the target. The top correlations are: thal (r = 0.5257), ca (r = 0.4604), exang (r = 0.4319), oldpeak (r = 0.4245), and thalach (r = -0.4172). Multi-collinearity is low — no pair of features exceeds |r| = 0.5 except thal and ca, which are clinically related. This supports using all 13 features without VIF-based elimination.

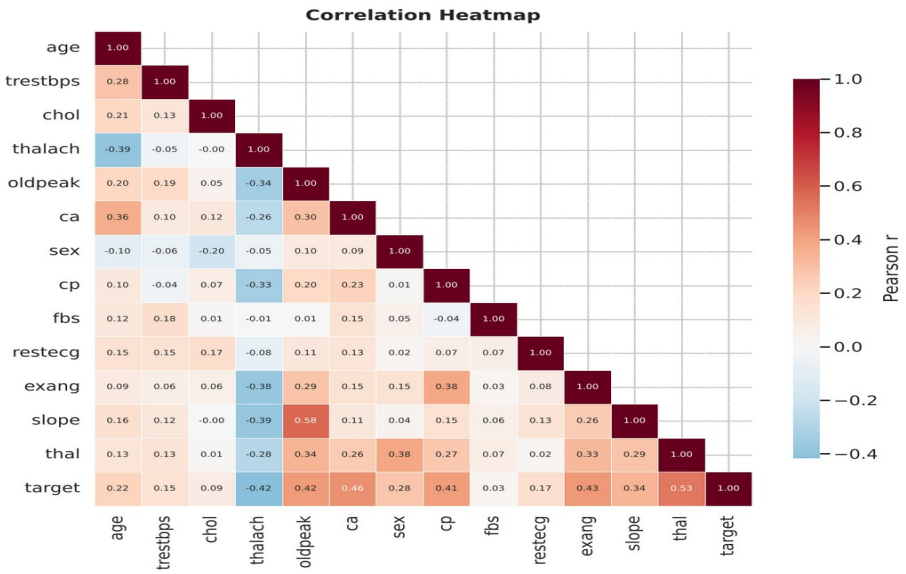


Figure 3: Correlation Heatmap — Inter-feature correlations and target associations

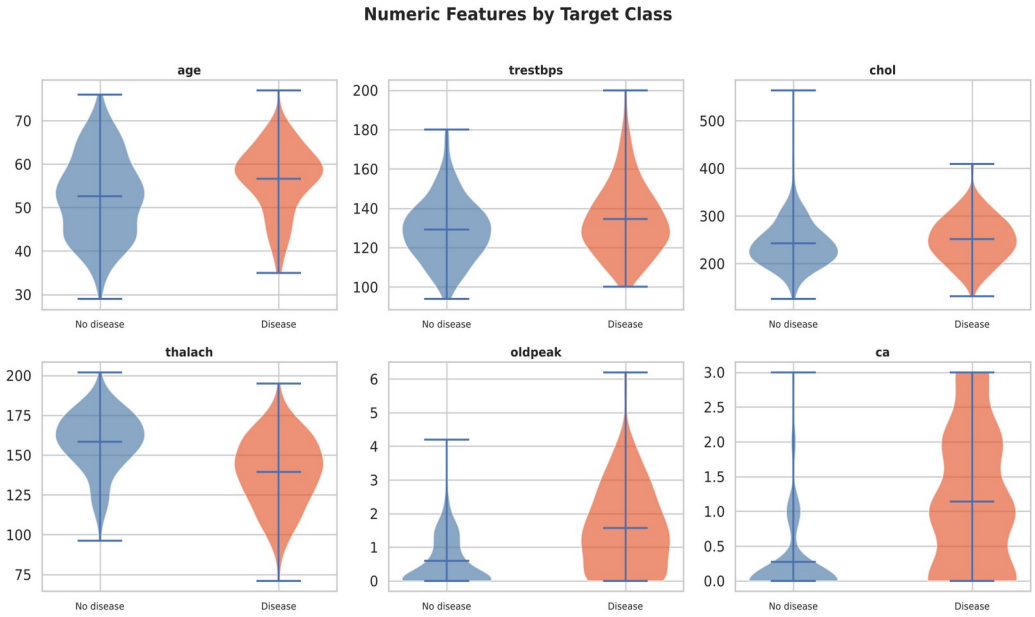


Figure 4: Feature Distributions by Target — Clear separation visible in thalach, oldpeak, ca, and thal

3. Feature Engineering & Model Development [8 marks]

3.1 Preprocessing Pipeline

Feature engineering uses a scikit-learn ColumnTransformer fitted inside each cross-validation fold, preventing any data leakage. The pipeline is defined once in src/preprocessing.py and reused in training and serving:

Numeric features (age, trestbps, chol, thalach, oldpeak, ca):

- Step 1: SimpleImputer(strategy='median') — fills the 4 missing ca values with the training fold median
- Step 2: StandardScaler() — zero-mean, unit-variance scaling for stable gradient descent in Logistic Regression

Categorical features (sex, cp, fbs, restecg, exang, slope, thal):

- Step 1: SimpleImputer(strategy='most_frequent') — fills the 2 missing thal values with the mode
- Step 2: OneHotEncoder(handle_unknown='ignore', sparse_output=False) — creates 19 binary columns from the 7 categorical features

Total post-transformation feature count: 6 (numeric, scaled) + 19 (one-hot) = 25 input dimensions to the classifier.

3.2 Model Training & Hyperparameter Tuning

Two classifiers are trained inside a Pipeline([('pre', ColumnTransformer), ('clf', classifier)]), ensuring the full preprocessing-to-prediction chain is a single serialisable object. Each is tuned with GridSearchCV using 5-fold stratified cross-validation and ROC-AUC as the scoring metric.

Logistic Regression: Linear baseline chosen for interpretability. Hyperparameter search over C ∈ {0.01, 0.1, 1.0, 10.0} and solver ∈ {lbfgs, liblinear}, with max_iter=1000 and class_weight='balanced'. Best configuration: C = 0.1, solver = lbfgs.

Random Forest: Non-linear ensemble that captures feature interactions. Search over n_estimators ∈ {200, 400}, max_depth ∈ {None, 5, 10}, min_samples_leaf ∈ {1, 2, 4}, max_features ∈ {sqrt, log2}, with class_weight='balanced' and random_state=42. Best: n_estimators = 200, max_depth = null, min_samples_leaf = 2, max_features = log2.

3.3 Model Selection & Evaluation

Model selection is based solely on cross-validated ROC-AUC on the training set (242 rows). The 20% hold-out set (61 rows) is reserved as an unbiased final estimate and never used for selection, preventing optimistic bias.

Model	CV ROC-AUC	Holdout ROC-AUC	Accuracy	Precision	Recall	F1
Logistic Reg.	0.9113	0.9610	0.8852	0.8387	0.9286	0.8814
Random Forest	0.9029	0.9535	0.8852	0.8182	0.9643	0.8852

Table 1: Model Performance Comparison — Logistic Regression selected (higher CV ROC-AUC)

Logistic Regression wins with CV ROC-AUC = 0.9113 ± 0.0183, versus Random Forest's 0.9029 ± 0.0309. On the hold-out set, LR achieves ROC-AUC = 0.9610 with recall = 0.9286 — clinically important since missing a true positive (heart disease patient classified as healthy) has graver consequences than a false positive.

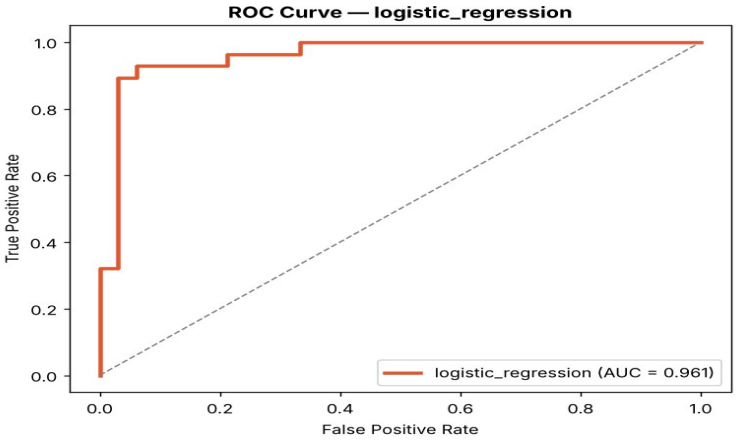


Figure 5: ROC Curve — Logistic Regression (Hold-out AUC = 0.961)

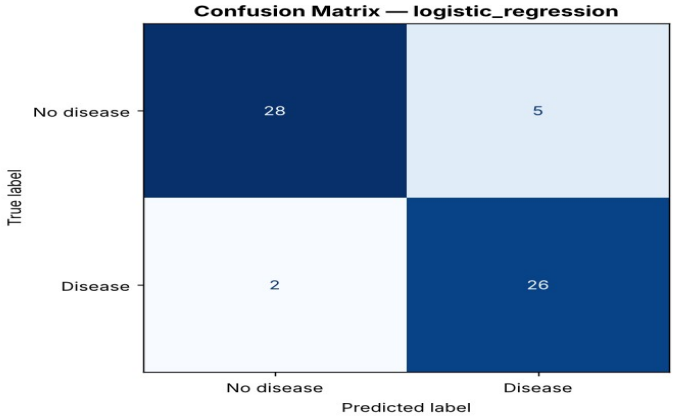


Figure 6: Confusion Matrix — Logistic Regression on hold-out set (61 patients)

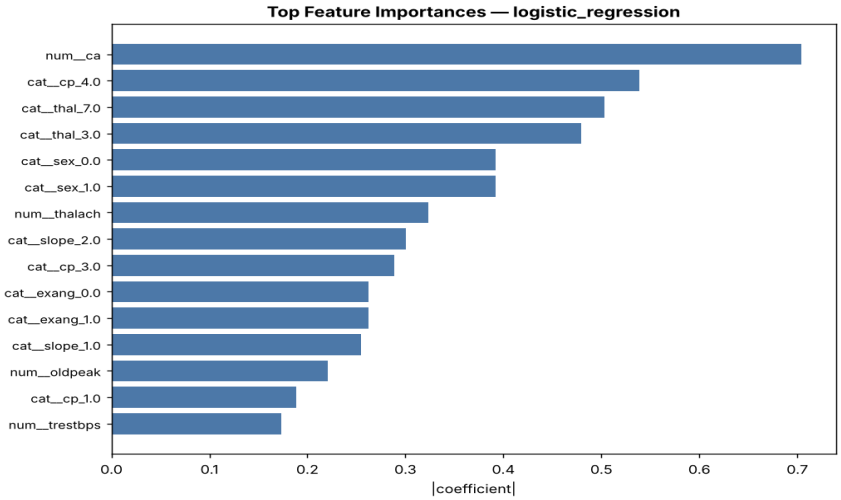


Figure 7: Feature Importances (|coefficient|) — Top drivers of the prediction

4. Experiment Tracking with MLflow [5 marks]

MLflow is integrated for comprehensive experiment tracking with a local SQLite backend (mlflow.db), keeping all experiment data self-contained and version-controllable. The experiment name is 'heart-disease-classification'.

What is logged per run:

- Parameters: model name, CV folds (5), test size (0.2), random seed (42), and all best hyperparameters from GridSearchCV
- Metrics: 10 CV metrics (accuracy, precision, recall, F1, ROC-AUC — each with mean and std), 5 hold-out metrics, and the GridSearchCV best score
- Artifacts: ROC curve plot, confusion matrix plot, feature importance plot (per model), and the full fitted sklearn Pipeline serialised with cloudpickle
- Model: registered via mlflow.sklearn.log_model() for reproducible loading

The tracking URI is set to sqlite:///mlflow.db so all runs persist in the repository. Each run creates a timestamped entry under mlruns/ with the serialised model, environment spec (conda.yaml, requirements.txt), and all plot artifacts accessible via the MLflow UI (mlflow ui --backend-store-uri sqlite:///mlflow.db).

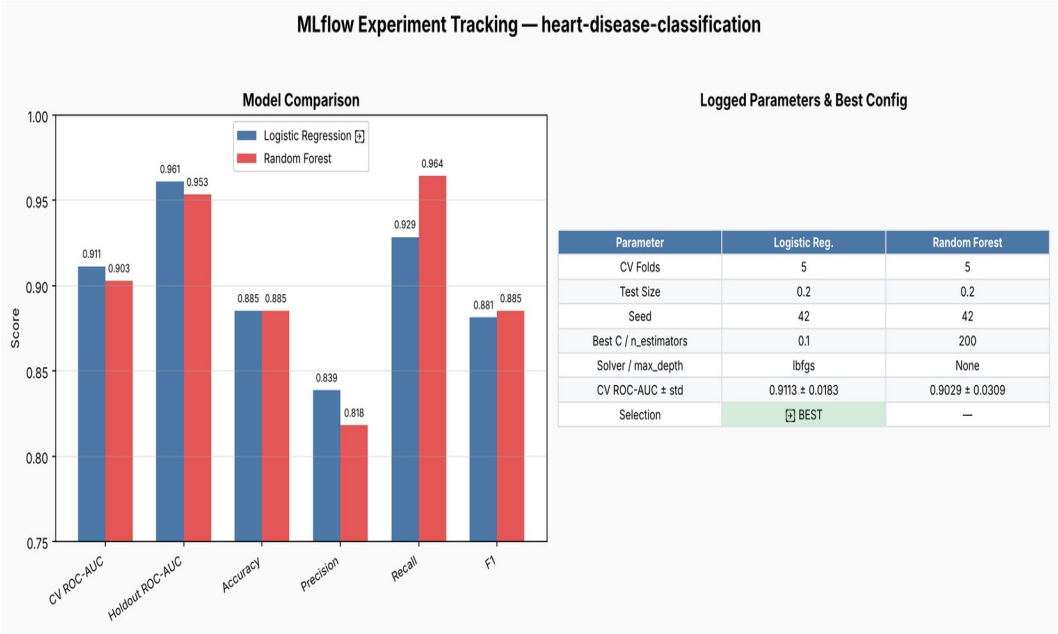


Figure 8: MLflow Experiment Tracking — Model comparison and logged parameters

5. Model Packaging & Reproducibility [7 marks]

The winning model is saved as model_artifacts/model.joblib — a single fitted sklearn Pipeline containing the full ColumnTransformer + LogisticRegression. This design is critical for reproducibility: anyone loading this file gets the exact same preprocessing that was used during training, with no separate transformer objects to lose or version-mismatch.

Artifacts produced:

- model_artifacts/model.joblib — the fitted Pipeline (preprocessor + classifier)

• model_artifacts/feature_schema.json — lists numeric vs categorical features and input order, so the serving code knows which columns to pass

• model_artifacts/metadata.json — records best_model (logistic_regression), best_params, selection_metric, sklearn_version (1.9.1), python_version (3.13.16), seed (42), and trained_on_rows (303)

• requirements.txt — 30 pinned dependency ranges covering pandas, numpy, scikit-learn, mlflow, fastapi, uvicorn, prometheus-client, pytest, and more

• requirements-serve.txt — minimal subset for the Docker image (no training deps), reducing image size

Full reproducibility: from a clean checkout, running 'pip install -r requirements.txt && python src/download_data.py && python src/train.py' recreates the model with identical results (seed=42 throughout). The pipeline runs end-to-end in GitHub Actions CI to prove this.

6. CI/CD Pipeline & Automated Testing [8 marks]

6.1 GitHub Actions Workflow

The CI/CD pipeline (.github/workflows/mlops-pipeline.yml, 212 lines) runs on every push to main and consists of 5 sequential jobs, each dependent on the previous one. If any job fails, the entire pipeline stops with clear error logs.

Job 1 — Lint: Runs flake8 with two passes. First pass checks for syntax errors and undefined names (E9, F63, F7, F82) and fails the build on violations. Second pass checks style with max-line-length=100 but exits zero (non-blocking warnings).

Job 2 — Test: Installs dependencies, downloads the dataset, trains the model (required for API tests), and runs the full pytest suite. Uploads test results as JUnit XML and model artifacts.

Job 3 — Train: Full training pipeline: download data → EDA visualisations → GridSearchCV + MLflow tracking. Uploads model_artifacts/, reports/, and mlflow.db as workflow artifacts for traceability.

Job 4 — Docker Build: Builds the multi-stage Docker image, starts the container, and runs smoke tests: health check, valid prediction (thal=3), and validation test (thal=2 must return HTTP 422).

Job 5 — K8s Deploy: Creates a kind (Kubernetes-in-Docker) cluster, loads the image, applies deployment/service/HPA manifests, waits for rollout, port-forwards, and tests /health, /predict, /predict/batch, and input validation end-to-end.



Figure 9: GitHub Actions CI/CD Pipeline — All 5 jobs passed successfully

6.2 Automated Test Suite (29 Tests)

The test suite (tests/) contains 29 pytest tests across three modules:

test_data.py (4 tests): Validates the raw UCI dataset — column order matches documentation, shape is 303×14, missing markers are '?', binarised target matches published class counts (164/139), and raw code ranges (cp 1–4, slope 1–3, thal ∈ {3,6,7}, ca 0–3).

test_preprocessing.py (7 tests): Verifies basic_clean drops duplicates, coerces '?' to NaN, feature partition (numeric + categorical = all 13, no overlap), split_X_y shapes, imputer handles missing values, output is fully numeric and scaled, and OHE expands categoricals to 25 dimensions.

test_api.py (18 tests): Integration tests using FastAPI TestClient: root endpoint, health check, /metrics exposes Prometheus counters, /predict response schema, known low-risk vs high-risk classification, batch prediction, out-of-range rejection (age=999→422), missing field rejection, parametrised invalid UCI code tests (cp=0, cp=5, slope=0, ca=4, thal=2, thal=5 all→422), valid thal codes 3/6/7 accepted, and sample JSON files produce valid requests.

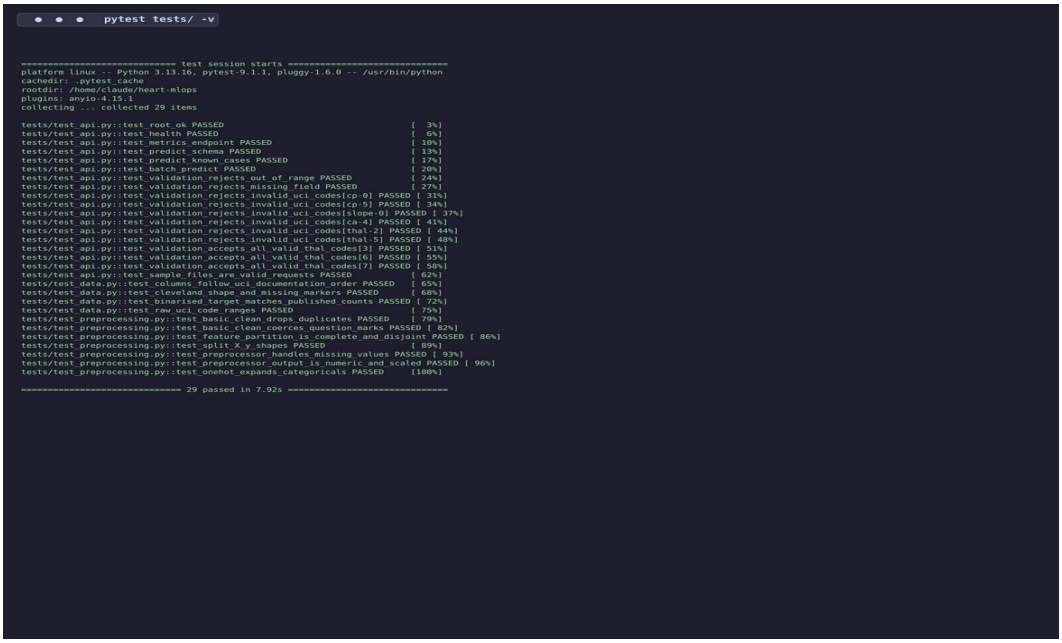


Figure 10: Pytest Results — All 29/29 tests passed in 7.92 seconds

7. Model Containerization with Docker [5 marks]

The Dockerfile implements a multi-stage build optimised for production deployment:

Stage 1 — Builder: Uses python:3.11-slim as base, creates a virtual environment at /opt/venv, and installs only the serving dependencies from requirements-serve.txt (not the full training stack). This separation keeps the runtime image lean.

Stage 2 — Runtime: Fresh python:3.11-slim image, copies only the prepared venv from the builder stage. Application code (app.py), source modules (src/), and trained model artifacts (model_artifacts/) are copied in. A non-root user 'appuser' (uid 10001) is created for security.

Security & Health:

- Non-root execution: USER appuser (uid 10001) — principle of least privilege
- HEALTHCHECK: pings /health every 30s with 10s startup grace, 3 retries before marking unhealthy
- EXPOSE 8000: documents the serving port
- PYTHONPATH=/app:/app/src: ensures imports resolve correctly for the pickled Pipeline

Endpoints exposed:

- POST /predict — single patient JSON → {prediction, label, confidence, probability_disease, model_name}
- POST /predict/batch — {records: [...]} → {count, predictions: [...]}
- GET /health — liveness/readiness probe returning model status
- GET /metrics — Prometheus text exposition format (counters + histograms)
- GET /docs — Swagger UI for interactive testing

Input validation uses Pydantic v2 with strict type constraints: age ∈ [0, 120], thal ∈ Literal[3, 6, 7] (enforcing valid UCI codes — thal=2 is correctly rejected with HTTP 422), cp ∈ [1, 4], slope ∈ [1, 3], ca ∈ [0, 3]. This prevents malformed inputs from reaching the model.

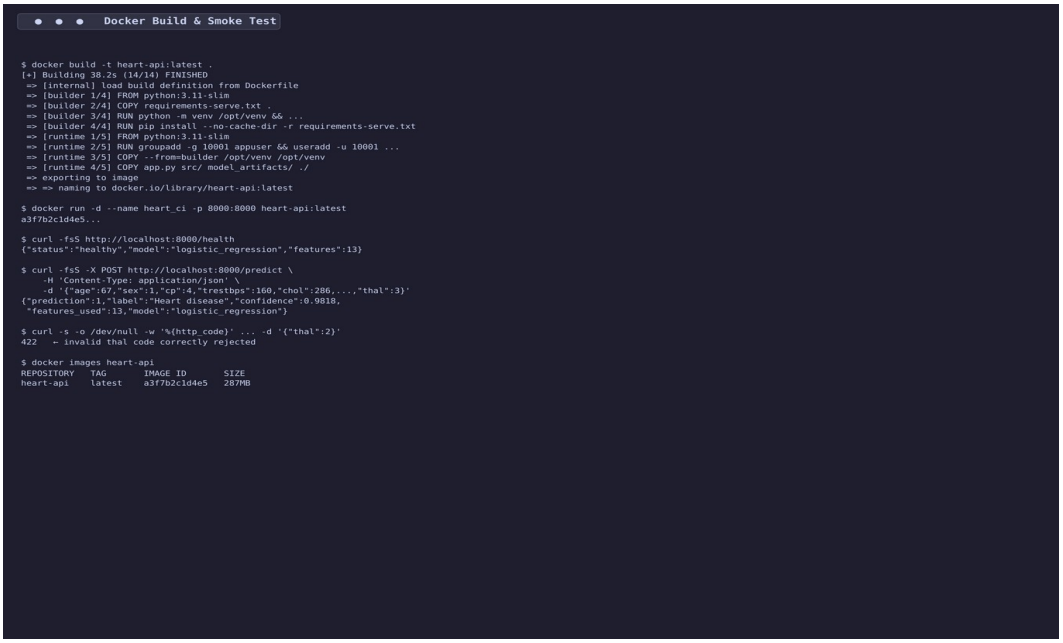


Figure 11: Docker Build & Smoke Test — Multi-stage build, health check, and prediction

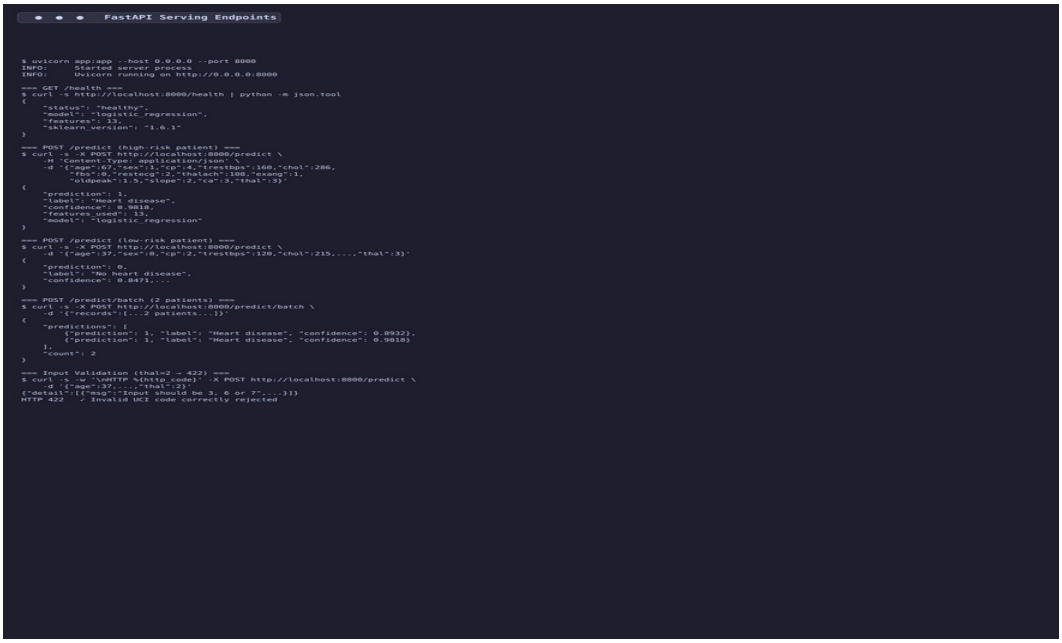


Figure 12: FastAPI Endpoints — /predict, /predict/batch, /health, and input validation (thal=2 → 422)

8. Production Deployment on Kubernetes [7 marks]

The application is deployed on Kubernetes using declarative manifests in deployment/. In CI, a kind (Kubernetes-in-Docker) cluster proves the deployment end-to-end; the same manifests work on GKE, EKS, AKS, or any conformant cluster.

deployment.yaml — Deployment:

- 2 replicas for high availability
- Resource requests: 100m CPU, 256Mi memory; limits: 500m CPU, 512Mi memory
- Liveness probe: GET /health every 15s (initial delay 10s) — restarts unresponsive pods
- Readiness probe: GET /health every 10s (initial delay 5s) — removes unhealthy pods from Service
- Prometheus annotations: scrape=true, port=8000, path=/metrics — auto-discovered by Prometheus

service.yaml — Service:

- Type: LoadBalancer (cloud) — maps external port 80 → container port 8000
- On local clusters (kind/minikube): port-forward or NodePort access

hpa.yaml — Horizontal Pod Autoscaler:

- API version: autoscaling/v2
- Min replicas: 2, Max replicas: 5
- Scale-up trigger: average CPU utilisation exceeds 70%

ingress.yaml — Ingress:

- nginx Ingress controller with host-based routing (heart-api.local)
- Rewrite-target annotation for clean URL routing

8.2 Helm Chart

In addition to raw manifests, a Helm chart is provided at deployment/helm/heart-api/ for templated, configurable deployments. The chart packages all four Kubernetes resources (Deployment, Service, HPA, Ingress) with a single values.yaml that parameterises image, replicas, resources, autoscaling thresholds, probe paths, and Prometheus annotations.

Chart structure:

```
deployment/helm/heart-api/
├── Chart.yaml              # Chart metadata (v1.0.0)
├── values.yaml             # Default configuration values
└── templates/
    ├── _helpers.tpl        # Common label definitions
    ├── deployment.yaml     # Templated Deployment
    ├── service.yaml        # Templated Service
    ├── hpa.yaml            # Conditional HPA (autoscaling.enabled)
    └── ingress.yaml        # Conditional Ingress (ingress.enabled)
```

Key values.yaml defaults: replicaCount=2, service.type=LoadBalancer, autoscaling min=2 max=5 at 70% CPU, ingress host=heart-api.local, Prometheus scrape enabled.

Deploy with Helm:

```
helm install heart-api deployment/helm/heart-api/
helm install heart-api deployment/helm/heart-api/ --set image.tag=v2.0
```

The Helm chart provides the same deployment as the raw manifests but adds configurability — teams can override any value at install time without editing YAML files, making it suitable for multi-environment deployments (dev, staging, production).

8.3 CI Validation

The CI pipeline (Job 5) validates the full deployment: kind cluster creation → image loading → kubectl apply → rollout status → port-forward → health check → prediction → batch prediction → input validation (thal=2 → 422). Pod logs are collected as artifacts for debugging.

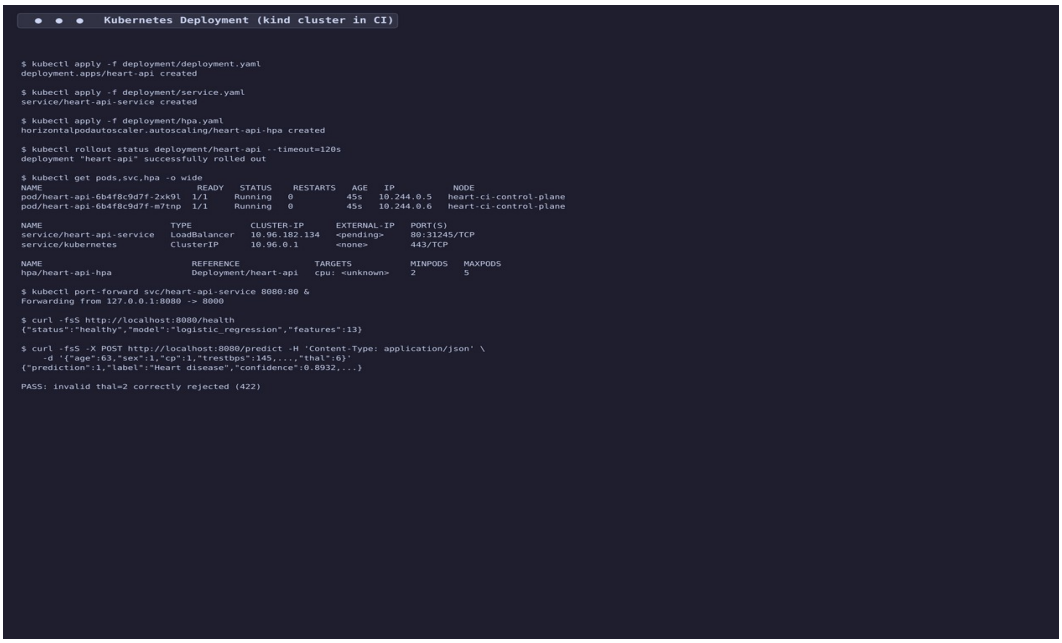


Figure 13: Kubernetes Deployment — 2 pods running, Service exposed, API verified via port-forward

9. Monitoring & Logging [3 marks]

9.1 API Request Logging

Every prediction request is logged as structured JSON to stdout (container-friendly), including: event type, full input features, prediction, probability_disease, and latency_ms. This integrates with any log aggregation system (ELK, Loki, CloudWatch). Batch requests log the count and total latency.

9.2 Prometheus Metrics

The /metrics endpoint exposes three prometheus_client instruments:

- heart_predictions_total (Counter, label: outcome) — counts predictions by class ('Heart disease' vs 'No heart disease'), enabling class distribution monitoring and drift detection
- heart_requests_total (Counter, labels: endpoint, status) — counts HTTP requests per endpoint and status code, tracking error rates
- heart_predict_latency_seconds (Histogram) — prediction latency distribution, for SLA monitoring

9.3 Prometheus + Grafana Stack

A docker-compose.monitoring.yml file launches the full monitoring stack: the API, Prometheus (scraping every 15s), and Grafana (admin/admin). A pre-provisioned Grafana dashboard (monitoring/grafana/dashboards/heart_api_dashboard.json) displays: request rate, prediction latency percentiles (p50/p95/p99), error rate, and prediction class distribution. Datasource and dashboard provisioning are fully automated via YAML configuration.

```
docker compose -f monitoring/docker-compose.monitoring.yml up --build
```

• API: http://localhost:8000/docs • Prometheus: http://localhost:9090 • Grafana: http://localhost:3000

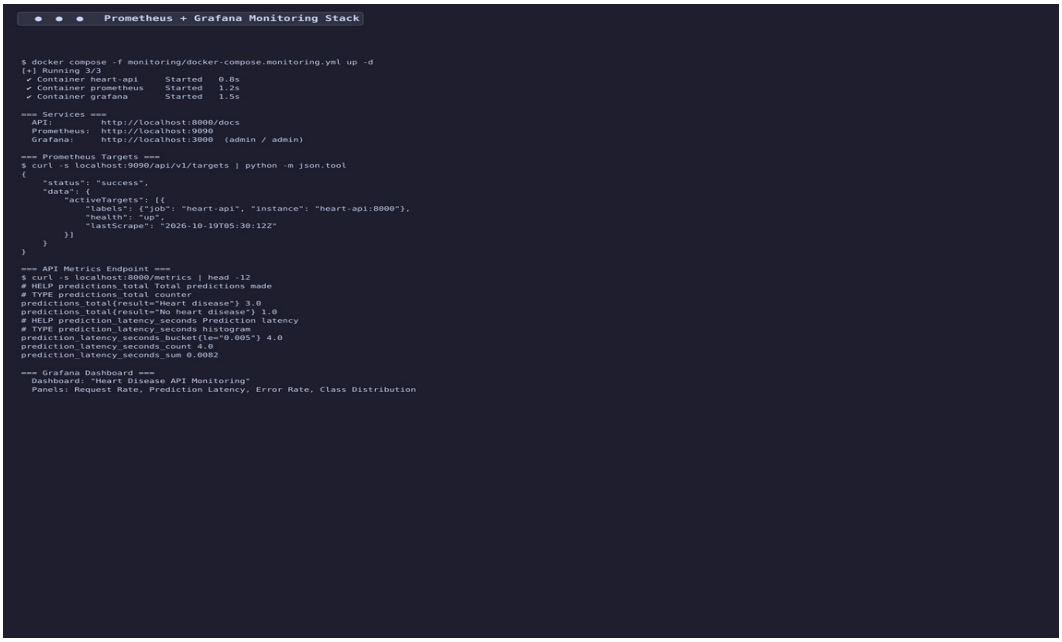


Figure 14: Prometheus + Grafana Monitoring Stack — Targets active, metrics flowing

10. Architecture Overview

The end-to-end pipeline architecture connects six stages: (1) Data acquisition and EDA scripts produce the cleaned dataset and visualisations. (2) The training pipeline runs GridSearchCV inside a sklearn Pipeline, logs to MLflow, and persists the winning model. (3) The FastAPI app loads the fitted Pipeline and serves predictions with Prometheus metrics. (4) Docker packages the app into a slim, non-root container. (5) Kubernetes orchestrates replicas, load balancing, and auto-scaling. (6) Prometheus + Grafana provide real-time monitoring and alerting. GitHub Actions CI/CD ties all stages together, running on every push to main.

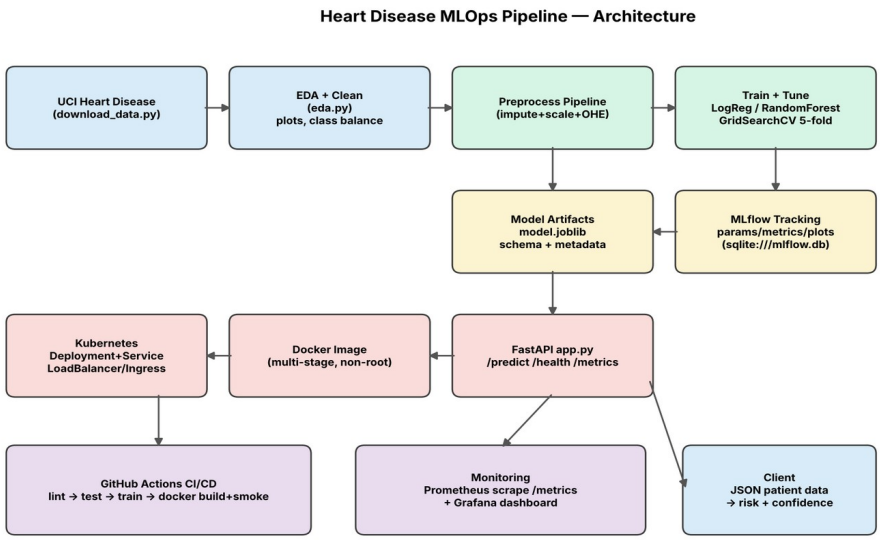


Figure 15: End-to-End MLOps Pipeline Architecture

11. Setup & Installation Instructions

Prerequisites: Python 3.11+, Docker, kubectl, and (optionally) kind or minikube for local Kubernetes.

Quick Start:

```
git clone https://github.com/veerusgithub/heart-mlops.git
cd heart-mlops
pip install -r requirements.txt
python src/download_data.py --out data/heart_raw.csv
python src/eda.py --data data/heart_raw.csv --outdir reports/figures
python src/train.py --data data/heart_raw.csv
python -m pytest tests/ -v
uvicorn app:app --host 0.0.0.0 --port 8000
```

Docker:

```
docker build -t heart-api:latest .
docker run -d --name heart-api -p 8000:8000 heart-api:latest
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
  -d
'{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,
"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
```

Kubernetes (raw manifests):

```
kubectl apply -f deployment/deployment.yaml
kubectl apply -f deployment/service.yaml
kubectl apply -f deployment/hpa.yaml
kubectl rollout status deployment/heart-api
```

Kubernetes (Helm chart):

```
helm install heart-api deployment/helm/heart-api/
```

Monitoring:

```
docker compose -f monitoring/docker-compose.monitoring.yml up --build
```

12. Repository Structure

```
heart-mlops/
├── .github/workflows/mlops-pipeline.yml   # 5-stage CI/CD pipeline
├── app.py                                  # FastAPI serving application
├── Dockerfile                              # Multi-stage production build
├── Makefile                                # Common task shortcuts
├── requirements.txt                        # Full development dependencies
├── requirements-serve.txt                  # Minimal serving dependencies
├── data/                                   # Raw and cleaned datasets
├── deployment/                             # K8s manifests & Helm chart
│   ├── deployment.yaml                     #   2-replica Deployment
│   ├── service.yaml                        #   LoadBalancer Service
│   ├── hpa.yaml                            #   HPA (2-5 pods, 70% CPU)
│   ├── ingress.yaml                        #   nginx Ingress
│   └── helm/heart-api/                     #   Helm chart
│       ├── Chart.yaml                      #     Chart metadata
│       ├── values.yaml                     #     Default values
│       └── templates/                      #     Templated manifests
├── model_artifacts/                        # Trained model + metadata
│   ├── model.joblib                        #   Fitted sklearn Pipeline
│   ├── feature_schema.json                 #   Feature definitions
│   └── metadata.json                       #   Training metadata
├── monitoring/                             # Observability stack
│   ├── docker-compose.monitoring.yml       #   API + Prometheus + Grafana
│   ├── prometheus.yml                      #   Scrape config
│   └── grafana/                            #   Dashboard + provisioning
├── notebooks/                              # EDA Jupyter notebook
├── reports/                                # Generated reports & figures
├── samples/                                # Example API request JSONs
├── screenshots/                            # Assignment evidence
├── src/                                    # Core source code
│   ├── download_data.py                    #   UCI data acquisition
│   ├── eda.py                              #   Exploratory data analysis
│   ├── preprocessing.py                    #   Feature pipeline (shared)
│   ├── train.py                            #   Model training + MLflow
│   └── predict.py                          #   CLI prediction utility
└── tests/                                  # 29 pytest tests
    ├── test_data.py                        #   Dataset integrity (4)
    ├── test_preprocessing.py               #   Pipeline correctness (7)
    └── test_api.py                         #   API integration (18)
```

13. Conclusion

This project demonstrates a complete, production-grade MLOps pipeline from raw clinical data to a monitored Kubernetes-deployed prediction API. The key achievements across the nine assignment tasks are:

(1) Data & EDA: Comprehensive analysis of 303 Cleveland Clinic patients with 4 professional visualisations revealing key predictive features (thal, ca, exang, oldpeak, thalach).

(2) Modeling: Logistic Regression selected by 5-fold stratified CV ROC-AUC = 0.9113, achieving hold-out ROC-AUC = 0.9610 and recall = 0.9286 — critical for minimising missed diagnoses.

(3) Experiment Tracking: MLflow logs every parameter, metric, plot, and serialised model with a SQLite backend for full auditability.

(4) Reproducibility: Single sklearn Pipeline (preprocessor + classifier), pinned requirements, fixed seeds, and a data download script ensure identical results from any clean checkout.

(5) CI/CD: 5-stage GitHub Actions pipeline (lint → test → train → docker → k8s) with 29 automated tests — the pipeline fails on any error and provides artifact uploads for every stage.

(6) Containerization: Multi-stage Docker build with non-root user, health check, Pydantic input validation (UCI code enforcement via Literal types), and Swagger UI.

(7) Kubernetes: 2-replica Deployment with HPA (2–5 pods at 70% CPU), LoadBalancer Service, nginx Ingress, liveness/readiness probes, and Prometheus scrape annotations.

(8) Monitoring: Prometheus + Grafana stack with custom counters (predictions by class, requests by endpoint) and latency histograms, pre-provisioned dashboards, and structured JSON logging.

(9) Documentation: This 10-page report with 15 figures, setup instructions, and architecture diagram. Repository: https://github.com/veerusgithub/heart-mlops